

Communication & Performance Tutorial — Distributed CG on AMD GPUs

A step-by-step tutorial built around the two examples in this repository:

- **CG-CPU/** — a distributed, CPU-only Conjugate Gradient (CG) solver (reference implementation).
- **CG-GPU/** — the same solver ported to AMD GPUs (rocSPARSE / rocBLAS / RCCL) with **five interchangeable communication variants** for the halo exchange.

The goal is not just to run the examples, but to walk the **path to a trustworthy, optimized performance measurement** so you can fairly compare communication approaches on AMD Instinct MI300A hardware.

Who this is for

You are comfortable with C++/MPI and have access to an MI300A node (or similar CDNA3 APU) with ROCm and a GPU-Aware MPI. You want to understand how to measure GPU communication cost correctly and which transport to pick.

The path (read in order)

#	Chapter	What you get
0	Introduction & setup	The problem, the two examples, hardware/software prerequisites
1	The CPU reference	Build/run <code>CG-CPU</code> , understand the algorithm and distributed SpMV
2	Porting to the GPU	Build/run <code>CG-GPU</code> , what changed (rocSPARSE/rocBLAS/RCCL/GPU-Aware MPI)
3	Measuring performance correctly	The core chapter: affinity, reproducibility, warm-up, repeats, isolating comm vs compute
4	Comparing communication variants	The five transports, a fair comparison, and how to read the results
5	Tuning the copy engines (SDMA vs blit)	<code>HSA_ENABLE_SDMA</code> / <code>_GANG</code> , when each wins
6	ROCm version & the SpMV API	Version sweeps, the 7.x compute regression, <code>rocsparse_v2_spmv</code> , OpenMPI vs cray-mpich
7	The optimized configuration	Putting it together: a recommended launch, a checklist, and where to go next
8	Profiling communication & compute	The system profilers — rocprofv3 (incl. ATT), rocprof-compute, rocprofiler-systems, TAU, HPCToolkit, likwid, AMD uProf, IntelliKit, roofline-extractor, rocBudAI, Linux perf, cachegrind — and perf_events security

#	Chapter	What you get
9	perf_events security — implementation & demo	The <code>perf_event_paranoid</code> ladder, <code>CAP_PERFMON</code> , resource limits; a no-root demo on <code>CG-CPU</code> (kernel-level, ROCm-independent)

TL;DR — the recommended optimized measurement

If you only remember one thing: **fix affinity and the RHS seed first, then take the minimum of several warm runs, and always split the reported time into communication vs. compute.** Everything else (which transport, which copy engine, which ROCm version) is a second-order comparison that is only meaningful once the measurement itself is under control.

```
cd CG-GPU
make
# reproducible, affinity-bound, comm/compute split reported:
CG_SEED=12345 mpirun -n 4 ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm isend
```

The two in-depth results write-ups this tutorial draws on:

- [CG-GPU/STUDY_REPORT.md](#) — AAC6 / MI300A Ubuntu, OpenMPI, ROCm 6.4.1–7.13.
- [CG-GPU/STUDY_REPORT_PrgEnv-amd.md](#) — HPE Cray EX, cray-mpich, ROCm 7.0.3.

0. Introduction & setup

The problem

Both examples solve the same linear system $\mathbf{A} \mathbf{x} = \mathbf{b}$ with the **Conjugate Gradient (CG)** method, where $\mathbf{A}$ is a large sparse symmetric positive-definite (SPD) matrix. The test matrix `Dubcova2.pm` is $65\,536 \times 65\,536$ and SPD. The matrix is distributed across MPI ranks by rows; each rank owns a contiguous block of rows and the corresponding slice of the solution vector.

CG is an iterative solver. Each iteration needs:

1. one **sparse matrix-vector product** (SpMV), $\mathbf{A} \mathbf{p}$ — this is where inter-rank **communication** happens, because a rank needs "ghost" values of $\mathbf{p}$ owned by its neighbours;
2. two **dot products** ($\mathbf{r} \cdot \mathbf{r}$ and $\mathbf{p} \cdot \mathbf{A} \mathbf{p}$) — each requires an `MPI_Allreduce` across all ranks;
3. a few **vector updates** (`axpy`, `scal`) — purely local.

So a distributed CG iteration has exactly two kinds of communication:

- **halo exchange** (neighbour-to-neighbour) inside the SpMV, and
- **global reductions** (`MPI_Allreduce`) for the dot products.

This tutorial is about measuring and optimizing the **halo exchange** — the part where you get to choose the transport — while keeping everything else fixed.

The two examples

```
cg-solver-example/  
├─ CG-CPU/      reference: plain C++/MPI, std::vector, hand-written loops (no GPU)  
└─ CG-GPU/      GPU port: rocSPARSE / rocBLAS / RCCL + GPU-Aware MPI, 5 comm variants
```

`CG-GPU/` is a line-by-line evolution of `CG-CPU/`: same algorithm, same matrix, same MPI comm pattern setup. Only the numerical kernels move to the GPU and the halo exchange gains four alternative implementations. That shared lineage is what makes the comparison meaningful.

The five communication variants (preview)

method	transport	buffers	collective?
<code>staged</code>	<code>MPI_Isend/Irecv</code>	host (D→H, H→D copies)	no
<code>isend</code>	<code>MPI_Isend/Irecv</code>	GPU (GPU-Aware MPI)	no

method	transport	buffers	collective?
rccl	ncclSend/ncclRecv	GPU (RCCL)	no
alltoallv_staged	MPI_Alltoallv	host	yes
alltoallv	MPI_Alltoallv	GPU (GPU-Aware MPI)	yes

All five produce **identical numerical results** — they differ only in how ghost values move.

Hardware & software prerequisites

Hardware: an AMD Instinct **MI300A** node (this tutorial's reference). MI300A is an APU: CPU + GPU share HBM, and each of the 4 (or 8, depending on partitioning) GPU dies is local to one NUMA node. The affinity lessons in Chapter 3 apply to any multi-GPU AMD node.

Software:

- **ROCm** $\geq$ 6.3 (provides `hipcc`, `rocSPARSE`, `rocBLAS`, `RCCL`). This tutorial shows results across 6.4.1–7.13.
- A **GPU-Aware MPI**: either OpenMPI/UCX built `--with-rocm`, or HPE `cray-mpich` with `craype-accel-amd-gfx942`.
- A batch scheduler (SLURM) for reproducible, exclusive allocations.

Two module setups appear throughout:

```
# OpenMPI + ROCm (AAC6 / MI300A Ubuntu)
module load rocm/6.4.1
module load openmpi/5.0.10-uccl.6.0-ucx1.19.1-xpmem-2.7.4

# HPE Cray EX + cray-mpich (PrgEnv-amd)
module swap PrgEnv-cray PrgEnv-amd/8.7.0
module load craype-accel-amd-gfx942
```

The `CG-GPU/Makefile` auto-detects which of these you are in (it keys off the Cray `PE_ENV`), so `make` works in both without edits.

What "done" looks like

By the end you will be able to produce a table like this — and, more importantly, **trust it**:

method	solve (s)	comm (s)	compute (s)	comm %
staged	0.148	0.124	0.023	84 %
isend	0.051	0.028	0.023	55 %
rccl	0.051	0.025	0.025	50 %

method	solve (s)	comm (s)	compute (s)	comm %
alltoallv_staged	0.082	0.056	0.026	68 %
alltoallv	0.051	0.025	0.026	49 %

(AAC6, ROCm 6.4.1, 4 ranks, `CG_SEED=12345` . Your absolute numbers will differ; the methodology is what transfers.)

Next: [1. The CPU reference →](#)

1. The CPU reference (`CG-CPU/`)

Start here even if you only care about the GPU. The CPU version is small, has no GPU or math-library moving parts, and establishes the algorithm and the distributed communication pattern that the GPU version inherits unchanged.

Build & run

```
cd CG-CPU
module load openmpi/5.0.10-ucc1.6.0-ucx1.19.1-xpmem-2.7.4 # any MPI with mpicxx
make
mpirun -n 4 ./cg_cpu src/Dubcova2.pm
```

Expected output:

```
172 iterations to converge
2-norm of residual: 0.000190
CG solve time:      0.1588 s (0.0009 s/iter)
```

The iteration count and residual will wobble slightly run-to-run because the right-hand side `b` is built from a **random** `x0`. Hold that thought — reproducibility is the subject of Chapter 3.

What's in `cg.cpp`

function	role
<code>spmv(Mat&, ...)</code>	serial CSR sparse matrix-vector product (single block)
<code>spmv(ParMat&, ...)</code>	distributed SpMV: post receives, pack+send, on-proc SpMV, wait, off-proc SpMV
<code>inner_product</code>	local dot product + <code>MPI_Allreduce</code>
<code>axpy / scale</code>	<code>x += α y / x *= α</code> (local loops)
<code>main</code>	read matrix, form comm pattern, run CG

The distributed SpMV — the pattern that matters

This is the halo exchange, and its structure is identical in every GPU variant later:

1. Post non-blocking receives for ghost values (`MPI_Irecv`)
2. Pack and post non-blocking sends (`loop + MPI_Isend`)
3. Compute the on-proc SpMV (overlaps with in-flight messages)
4. Wait for receives (`MPI_Waitall`)

5. Compute the off-proc SpMV using the ghosts
6. Wait for sends (MPI_Waitall)

Two ideas are baked in here and carry through the whole tutorial:

- **The matrix is split into an on-proc block and an off-proc block.** The on-proc block only touches locally owned vector entries, so it can be computed while messages are in flight. The off-proc block needs the ghost values, so it runs after the receives complete. This on-proc/off-proc split is what makes communication/computation **overlap** possible.
- **The comm pattern is set up once.** `sparse_mat.hpp` (`form_comm`) figures out, per rank, exactly which indices to send to which neighbour and how many to receive. This setup is done on the CPU and is identical in `CG-GPU/` — only the data movement changes.

The CG algorithm

```
r0 = b - A x0
p0 = r0
for i = 0, 1, 2, ...:
    α = (ri, ri) / (A pi, pi)    ← needs A pi (SpMV) and two dot products
    x += α pi
    r = b - A x                    ← residual recomputed every 8 iters for stability
    β = (ri+1, ri+1) / (ri, ri)
    p = r + β p
```

Per iteration: **1 SpMV** (halo exchange) + **2 dot products** (`MPI_Allreduce`) + a few local vector ops. That fixed structure is the "denominator" of every measurement later — the work per iteration never changes, so if the iteration count is held constant (Chapter 3), differences in solve time come only from the transport and the hardware/software stack.

Why keep the CPU version around?

- It is the **correctness oracle**: the GPU variants must converge to the same residual in the same number of iterations (for a fixed seed).
- It is a **baseline** for "how much did the GPU actually buy us?" — and a reminder that for a small per-rank problem the answer is subtle (the CPU solve here, ~0.16 s, is in the same ballpark as some unoptimized GPU launches, which is exactly why measurement discipline matters).

Next: [2. Porting to the GPU →](#)

2. Porting to the GPU (`CG-GPU/`)

`CG-GPU/` keeps the algorithm and the MPI comm-pattern setup from `CG-CPU/` and moves every numerical operation to the GPU. The two headers (`sparse_mat.hpp` , `par_binary_IO.hpp`) are the same — file I/O and comm setup stay on the host.

What changed

CG-CPU/	CG-GPU/
<code>std::vector<double></code>	<code>hipMalloc</code> / <code>hipMemcpy</code> device arrays
hand-written SpMV loop	<code>rocsparse_spmv</code> (rocSPARSE generic API)
hand-written pack/gather	<code>rocsparse_dgthr</code> (GPU gather)
hand-written dot	<code>rocbblas_ddot</code>
hand-written axpy/scal	<code>rocbblas_daxpy</code> / <code>rocbblas_dscal</code>
<code>MPI_Isend/Irecv</code> on host buffers	choice of 5 communication variants

Build & run

```
cd CG-GPU
module load rocm/6.4.1
module load openmpi/5.0.10-uccl.6.0-ucx1.19.1-xpmem-2.7.4
make
mpirun -n 4 ./cg_gpu src/Dubcova2.pm isend
```

The `Makefile` auto-detects the toolchain (OpenMPI+ROCm vs Cray PE) and the GPU arch. Useful knobs:

```
make GPU_ARCH=gfx90a  # target a specific arch (default: auto-detected, fallback gfx942)
make SPMV_V2=1        # use rocsparse_v2_spmv (ROCm 7.x); see Chapter 6
make run              # quick 4-rank sanity run over four methods
```

A first run prints something like:

```
method=isend ranks=4 gpus_visible=1
RHS seed: 12345 (CG_SEED)
Initial residual: 195.873
174 iterations to converge
2-norm of residual: 0.000188555
CG solve time:      0.0513 s (0.0003 s/iter)
  comm total:       0.0282 s (55.0% of solve)
  halo exchange:    0.0142 s (0.0001 s/iter, 27.6%)
```



```
dot allreduce:    0.0140 s  (27.3%)
compute (rest):   0.0231 s  (45.0%)
```

That last block — the **communication breakdown** — is instrumentation added for this tutorial and is the key to everything that follows. We'll dissect it in Chapter 3.

The three ports you should actually look at

1. SpMV: a library call instead of a loop

```
// CPU:
for (int i = 0; i < A.n_rows; i++)
    for (int j = A.rowptr[i]; j < A.rowptr[i+1]; j++)
        b[i] += alpha * A.data[j] * x[A.col_idx[j]];

// GPU:
rocspmv(handle, rocspmv_operation_none,
        &alpha, A.descr, vec_x, &beta, vec_b, ...);
```

The rocSPARSE descriptor wraps the CSR arrays uploaded once with `hipMemcpy`. The on-proc and off-proc blocks each get their own descriptor, preserving the overlap structure from Chapter 1.

2. Halo exchange: the send buffer is packed on the GPU

```
// Pack the send buffer on the GPU – no host copy
rocspmv_dgthr(handle, send_size, d_x, d_sendbuf, d_send_idx, ...);
hipDeviceSynchronize();           // gather must finish before MPI reads d_sendbuf

// GPU-Aware MPI: hand GPU pointers straight to MPI
MPI_Isend(d_sendbuf + off, count, MPI_DOUBLE, dest, ...);
MPI_Irecv(d_recvbuf + off, count, MPI_DOUBLE, src, ...);
```

Without GPU-Aware MPI you would copy device→host before the send and host→device after the receive. The `staged` variant does exactly those copies; the `isend` variant skips them. Comparing the two (Chapter 4) is how you measure the value of GPU-Aware MPI.

3. Dot products: `rocblas_ddot` then one sync then `MPI_Allreduce`

```
rocblas_ddot(blas_handle, n, d_a, 1, d_b, 1, &local_sum);
hipDeviceSynchronize();           // make the scalar visible on the host
MPI_Allreduce(&local_sum, &global, 1, MPI_DOUBLE, MPI_SUM, comm);
```

Every dot product forces a device→host sync. Two dots per iteration means two synchronization points per iteration — a latency cost that shows up as `dot allreduce` in the breakdown and is a prime optimization target (Chapter 7).

One GPU per rank

```
hipSetDevice(rank % num_gpus);
```

Under `mpirun` this hands ranks 0–3 distinct GPUs. That is correct but not the whole story: **which** cores a rank runs on relative to its GPU is what makes or breaks performance. That is Chapter 3, and it is the single most important chapter in this tutorial.

Next: [3. Measuring performance correctly →](#)

3. Measuring performance correctly

This is the heart of the tutorial. Before you compare any two communication approaches, the measurement itself has to be trustworthy. Five things, in priority order:

1. **Affinity** — bind each rank to its GPU-local CPU cores. (Biggest effect by far.)
2. **Reproducibility** — fix the RHS seed so every run solves the same system.
3. **Warm-up & repeats** — discard the first run; report the minimum (or median) of several.
4. **Isolate comm vs compute** — never compare a single "solve time" number blind.
5. **Verify correctness** — same iteration count and residual, or the comparison is meaningless.

Get these wrong and you will "measure" the scheduler, the RNG, or a cold cache instead of the transport.

3.1 Affinity — the make-or-break factor

On MI300A each GPU is local to one NUMA node (**GPU i $\leftrightarrow$ NUMA node i**). If a rank runs on cores that are remote from its GPU — or worse, if all ranks are crammed onto a handful of shared threads — every kernel launch, `hipDeviceSynchronize`, and MPI progress call pays a latency penalty.

How bad? On AAC6, the exact same binary and matrix, 4 ranks:

launch	CPU affinity	staged	isend	rccl	alltoallv
<code>--bind-to none</code> , not exclusive	4 hwthreads shared by all ranks (remote)	9.27 s	10.58 s	7.39 s	8.46 s
<code>--exclusive</code> + <code>gpu_bind.sh</code>	each rank $\rightarrow$ its GPU-local NUMA (24 cores)	0.15 s	0.05 s	0.05 s	0.05 s

That is a **$\sim 100\times$ difference** — dwarfing every transport, copy-engine, or ROCm-version effect in the rest of this tutorial. **If you take away one thing, take this: fix affinity first.**

Do it with a wrapper

Two ready-made per-rank wrappers ship with `CG-GPU/`:

```
# numactl-based, 4-GPU NUMA binding (OpenMPI):
mpirun -n 4 ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm isend

# taskset-based, 8-GPU MI300A binding (used by run_test_7.13.sh):
mpirun -n 8 --bind-to none bash set_affinity_mi300a.sh ./cg_gpu src/Dubcova2.pm isend
```

Both set `ROCR_VISIBLE_DEVICES=<local_rank>` (each rank sees exactly its own GPU) and pin the rank to that GPU's local cores (`gpu_bind.sh` via `numactl --cpunodebind/--membind; set_affinity_mi300a.sh` via `taskset`). Use `--bind-to none` on the `mpirun/srun` side so the wrapper owns all pinning.

Verify it, don't assume it

```
mpicxx -O2 -std=c++17 check_affinity.cpp -o check_affinity \
-I$ROCM_PATH/include -L$ROCM_PATH/lib -lamdhip64
mpirun -n 4 ./check_affinity
```

This prints each rank's selected GPU **PCI bus id** and CPU mask and flags any two ranks that collide on a GPU.

Two affinity traps

- **srun without PMIx** launches each task as its own MPI world of size 1, so every task computes `rank % num_gpus = 0` and they **all pile onto GPU 0**. Use `srun --mpi=pmix` or a binding wrapper.
- **The cliff is a property of the launch, not the machine.** On an HPE Cray EX system an `--exclusive srun` allocation already binds each rank to a distinct, largely NUMA-local core set, so explicit binding adds only **~3-8 %** there (see [STUDY_REPORT_PrgEnv-amd.md](#) §3). Never assume; measure bound vs. unbound on your system once, then standardize on the bound launch.

3.2 Reproducibility — fix the RHS seed

By default the right-hand side is seeded from wall-clock time, so each run solves a slightly different system and converges in a different number of iterations. You cannot compare methods whose iteration counts differ.

Fix the seed:

```
CG_SEED=12345 mpirun -n 4 ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm isend # via env
mpirun -n 4 ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm isend 12345 # via argv[3]
```

The seed is echoed as `RHS seed: 12345` at the top of the output. With it fixed, **all five methods converge in exactly the same iteration count with the same residual** (e.g. 172 iterations, final residual 1.815e-4). Now a difference in solve time is a difference in transport, not in how much work was done.

3.3 Warm-up and repeats

The first run of a freshly built binary pays one-time costs: library initialization, JIT/code-object load, allocator warm-up, and RCCL communicator bring-up. These are not what you want to compare.

- **Discard the first run** (or treat it as warm-up).
- **Repeat 3-5 times** and report the **minimum** (cleanest, least noise) — or the **median $\pm$ spread** if you want to characterize variability. For the fast GPU-Aware variants the run-to-run spread is comparable to the method-to-method difference, so a single run can rank two methods backwards.

The sweep scripts already do this. For example `sweep_sdma.sh` keeps the minimum-solve run over `REPEATS`:

```
REPEATS=5 CG_SEED=12345 ... # every sweep script honours these
```

3.4 Isolate communication from compute

A single "solve time" number hides the thing you are trying to measure. The instrumented solver reports a breakdown (max across ranks, accumulated over the CG loop only):

```
CG solve time:      0.0513 s  (0.0003 s/iter)
comm total:         0.0282 s  (55.0% of solve)
halo exchange:      0.0142 s  (0.0001 s/iter, 27.6%) ← the part you choose the transport
for
dot allreduce:      0.0140 s  (27.3%)                ← MPI_Allreduce for the two dot
products
compute (rest):     0.0231 s  (45.0%)                ← rocSPARSE SpMV + rocBLAS +
launch overhead
```

- **halo exchange** = staging copies (D $\leftrightarrow$ H) + GPU gather/pack + the MPI/RCCL `send / recv / wait / alltoallv` calls. This is what differs between the five variants.
- **dot allreduce** = the global reductions. Independent of the halo transport.
- **compute (rest)** = `solve - comm total`. rocSPARSE/rocBLAS kernels + launch latency. Independent of the halo transport — so if it moves when you change only the transport, that is your noise floor talking.

Two definitions and a caveat you must keep in mind:

- The timers cover the **CG loop only** — matrix read, GPU upload, and setup are excluded (they are one-time).
- For the **overlap variants** (`isend`, `rccl`, `alltoallv`) the `halo exchange` figure is mostly the exposed wait: the on-proc SpMV runs concurrently with the in-flight messages, so this is **not** the raw wire time. It is the cost you actually pay, which is

what you want for choosing a transport — just don't mistake it for a bandwidth measurement.

3.5 Verify correctness every time

Before trusting a timing, check the run actually solved the problem: same **iteration count** and **final residual** as the CPU reference (for the same seed). The bundled `test_cg_gpu.sh` does this automatically — it builds, runs every variant, asserts convergence to a tolerance, and prints the timing summary; it exits 0 gracefully on a node with no GPU.

```
# inside a GPU allocation, with binding:
GPU_BIND=./gpu_bind.sh CG_SEED=12345 ./test_cg_gpu.sh
# or submit:
sbatch submit_cg_gpu_test.sbatch
```

The measurement recipe (checklist)

```
[ ] Exclusive allocation (own whole NUMA nodes)
[ ] Per-rank GPU+CPU binding wrapper (gpu_bind.sh / set_affinity_mi300a.sh)
[ ] Verified binding once with check_affinity
[ ] Fixed CG_SEED
[ ] Warm-up run discarded
[ ] 3–5 repeats, report min (or median ± spread)
[ ] Read the comm/compute breakdown, not just solve time
[ ] Confirmed identical iteration count + residual across methods
```

Only once every box is ticked are the comparisons in the next chapters meaningful.

Next: [4. Comparing communication variants →](#)

4. Comparing communication variants

With the measurement under control (Chapter 3), you can now compare the five transports fairly. The design of the examples makes this a set of clean, controlled experiments: pairs of variants that differ in exactly one thing.

The five variants and what each isolates

method	transport	buffers	pairs with...	to isolate
staged	Isend/Irecv	host	isend	the D↔H staging copies
isend	Isend/Irecv	GPU (GPU-Aware)	staged	value of GPU-Aware point-to-point
alltoallv_staged	Alltoallv	host	alltoallv	the D↔H staging copies (collective)
alltoallv	Alltoallv	GPU (GPU-Aware)	alltoallv_staged	value of GPU-Aware collective
rccl	ncclSend/Recv	GPU (RCCL)	isend / alltoallv	GPU-native vs GPU-Aware MPI

Two controlled contrasts fall out immediately:

- **staged vs isend** and **alltoallv_staged vs alltoallv** each isolate the **PCIe/staging round-trip** (device→host before the send, host→device after the receive) that GPU-Aware MPI removes.
- **isend vs alltoallv vs rccl** compares three GPU-resident transports head-to-head.

Run the comparison

```
cd CG-GPU && make
for m in staged isend rccl alltoallv_staged alltoallv; do
  echo "=== $m ==="
  CG_SEED=12345 mpirun -n 4 ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm $m
done
```

Or use the harness that builds once and sweeps all methods with warm-up + repeats:

```
sbatch submit_cg_gpu_test.sbatch          # or: GPU_BIND=./gpu_bind.sh ./test_cg_gpu.sh
```

Results (AAC6, ROCm 6.4.1, 4 ranks, seed 12345, min of runs)

Times are seconds, max across ranks, CG loop only:

method	solve	comm total	halo excl	dot allreduce	compute	comm %
staged	0.1477	0.1243	0.1095	0.0148	0.0234	84 %
isend	0.0513	0.0282	0.0142	0.0140	0.0231	55 %
rccl	0.0505	0.0252	0.0160	0.0092	0.0253	50 %
alltoallv_staged	0.0823	0.0560	0.0422	0.0138	0.0263	68 %
alltoallv	0.0512	0.0250	0.0129	0.0121	0.0262	49 %

How to read it

- **Staging is the dominant cost when present.** staged halo (0.110 s) vs isend (0.014 s) is a $\sim 7\times$ gap that is purely the D→H + H→D copies GPU-Aware MPI eliminates. alltoallv_staged (0.042 s) vs alltoallv (0.013 s) shows the same round-trip on the collective path. **Takeaway: if you have GPU-Aware MPI, use it — this is the single largest transport-level win.**
- **The three GPU-resident transports are within noise of each other** here (~ 0.013 – 0.016 s halo). On one node, 4 GPUs, over XGMI, isend, alltoallv, and rccl have no decisive winner. Do not over-interpret a 1 ms gap that is smaller than your run-to-run spread — this is exactly why Chapter 3 insists on repeats.
- **compute is essentially constant across methods** (~ 0.023 – 0.026 s). Good — it should be, because you changed only the transport. If your compute column moves a lot between methods, your measurement is noisy (revisit affinity/warm-up), not the transport.
- **dot allreduce is a real, transport-independent cost** (0.009–0.015 s, up to ~ 27 % of solve): two latency-bound scalar allreduces per iteration, each gated by a hipDeviceSynchronize. No choice of halo transport touches it — it is a separate optimization axis (Chapter 7).

The subtlety: overlap means "halo" is exposed wait, not wire time

For isend/rccl/alltoallv the on-proc SpMV runs while messages are in flight, so the measured halo exchange is the part of the transfer that couldn't be hidden behind compute. That is the right number for "which transport costs me less," but it means:

- a faster raw transport can show a larger exposed halo if it leaves less compute to overlap with, and
- at larger per-GPU problem sizes (more on-proc work to hide behind), the exposed halo of the overlap variants shrinks further relative to staged.

To measure raw wire time instead, you would disable overlap or use a profiler trace (Chapter 7).

Does the ranking hold on a different MPI stack?

Yes. On HPE Cray EX with **cray-mpich** (ROCM 7.0.3), the same qualitative picture holds — staging is expensive, the GPU-resident transports are comparable, and cray-mpich's GPU-Aware path is on par with (marginally better than) GPU-Aware OpenMPI:

method	solve	comm total	halo exch	dot allreduce	compute	comm %
staged	0.1722	0.1064	0.0862	0.0202	0.0657	62 %
isend	0.0973	0.0182	0.0146	0.0036	0.0791	19 %
rccl	0.0926	0.0129	0.0099	0.0030	0.0796	14 %
alltoallv_staged	0.1261	0.0547	0.0439	0.0107	0.0714	43 %
alltoallv	0.0971	0.0191	0.0156	0.0035	0.0779	20 %

(from [STUDY_REPORT_PrgEnv-amd.md](#) §4). Note the `comm %` is much lower here — not because communication got faster, but because compute got slower on ROCm 7.x. That cross-cutting effect is Chapter 6.)

Caveats

Single node, single matrix, single rank count, latency-bound regime. Absolute deltas between the fast variants are small; treat cross-method differences as **indicative**, and always report the measurement conditions with the numbers. Strong/weak scaling and larger per-GPU sizes (where a transport's bandwidth, not its latency, dominates) are the natural next experiments.

Next: [5. Tuning the copy engines \(SDMA vs blit\)](#) →

5. Tuning the copy engines (SDMA vs blit kernels)

Once you've chosen a GPU-Aware transport, there's a lower-level knob that changes how ROCm physically moves bytes between GPU memory and the fabric — and it can shift communication time by 20–30 % with **zero code changes**.

The two mechanisms

When GPU-Aware MPI (or the staging `hipMemcpy`) moves data, ROCm can use either:

mechanism	HSA_ENABLE_SDMA	how it works
SDMA engines	1 (default)	dedicated hardware DMA controllers move data independently of the shader engines — no compute resources consumed
Blit kernels	0	ROCm dispatches a small compute shader ("blit") that copies using the shader engines — no dedicated DMA hardware

A second flag gangs multiple SDMA engines onto one transfer:

```
export HSA_ENABLE_SDMA=1
export HSA_ENABLE_SDMA_GANG=1
```

Because this is purely a runtime setting, **the binary is identical across configurations** — you only change the environment. That makes it a clean controlled experiment: anything that moves is the copy path, and **compute should stay flat** (a built-in sanity check).

Run the sweep

Switch it by hand:

```
HSA_ENABLE_SDMA=1 mpirun -n 4 ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm isend # SDMA
(default)
HSA_ENABLE_SDMA=0 mpirun -n 4 ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm isend # blit
kernels
```

...or use the sweep harness (builds once, sweeps blit / sdma / sdma+gang across all methods, min of `REPEATS`):

```
sbatch --export=ALL,ROCM_VER=6.4.3,REPEATS=5 submit_sdma_sweep.sbatch
```

The maintainer's `run_test_7.13.sh` sweeps `HSA_ENABLE_SDMA ∈ {1,0}` for the GPU-Aware variants (`isend`, `alltoallv`) and labels each run (`sdma`) / (`blit_kernel`) in the log.

Results (ROCm 6.4.3, 4 ranks, seed 12345, min of 5 runs)

Halo-exchange time and its change vs. blit:

method	blit	sdma	sdma_gang	halo Δ vs blit
staged	0.1215	0.1116	0.1129	−8 %
isend	0.0154	0.0119	0.0113	−27 %
rccl	0.0143	0.0129	0.0129	−10 %
alltoallv_staged	0.0480	0.0418	0.0394	−18 %
alltoallv	0.0116	0.0140	0.0121	+21 % (see below)

(full solve/comm/compute table in [STUDY_REPORT.md](#) §5.2. The Cray/ROCm 7.0.3 sweep in [STUDY_REPORT_PrgEnv-amd.md](#) §5 shows the same pattern: `isend` −24 %, `alltoallv` −23 %.)

How to read it

- **SDMA helps the copy-heavy paths**, exactly as expected. Disabling it (blit) is slower on `staged` (−8 %), `alltoallv_staged` (−13...−18 %), and GPU-Aware `isend` (−23...−27 %). Blit copies burn CU/shader cycles and contend with the SpMV kernels; SDMA offloads them to dedicated hardware.
- **Ganging adds a small extra win** on the copy-bound variants (`isend` −23 %→−27 %, `alltoallv_staged` −13 %→−18 %) and is within noise elsewhere.
- **Compute stays flat** across all three settings — the effect is cleanly isolated to data transport, which is the confirmation that you're measuring the copy path and not something else.
- **The one exception** (`alltoallv` marginally better with blit) shows the general rule: for very small, latency-sensitive transfers, the SDMA engine's start-up cost can exceed a quick shader copy, especially on MI300A's unified memory. The effect is small and near the noise floor.

When does each win?

- **SDMA** wins for **larger / copy-heavy** transfers where keeping the shader engines free to overlap with compute matters more than raw copy latency.
- **Blit kernels** can win for **small, latency-sensitive** transfers where the shaders are otherwise idle and the lower dispatch overhead beats the DMA engine's start-up.

The right choice is workload- and hardware-dependent, which is the real lesson: **it's a knob to sweep, not a constant to assume**. The default (`HSA_ENABLE_SDMA=1`) is the right starting point; forcing blit costs up to ~25 % of halo time on the staged and GPU-Aware point-to-point paths. Turn on `HSA_ENABLE_SDMA_GANG=1` when the halo volume is large.

Next: [6. ROCm version & the SpMV API](#) →

6. ROCm version & the SpMV API

The transport and copy-engine chapters optimize **communication**. But the comm/compute breakdown from Chapter 3 has a second column — **compute** — and it turns out to be the dominant, and most surprising, part of the story at this problem size. This chapter is a worked example of using the same measurement discipline to chase down a compute-side regression, and of not being fooled by a plausible-sounding fix.

Sweeping ROCm versions

Because the binary must be rebuilt per toolchain, the sweep harness rebuilds `cg_gpu` under each module and runs one method with warm-up + repeats, all in one exclusive allocation (so results are directly comparable):

```
sbatch --export=ALL,VERSIONS="6.4.1 7.0.2 7.2.4 7.13.0",REPEATS=5,METHOD=isend \
  submit_rocm_sweep.sbatch
```

Result: a compute regression at the 6.x → 7.x boundary

`isend`, 4 ranks, seed 12345, min of 5 runs (compute = solve – comm):

ROCm	solve (s)	comm (s)	compute (s)
6.4.1	0.0480	0.0178	0.0302 (baseline)
7.0.2	0.0717	0.0148	0.0569
7.2.4	0.0700	0.0141	0.0559
7.13.0	0.0713	0.0146	0.0567

Two things jump out:

- **Communication is as good or better on 7.x** (comm 0.018 → 0.014 s) — the MPI/UCX path did not regress.
- **Compute roughly doubled** (0.030 → 0.056 s) and is **flat across the whole 7.x line**, i.e. introduced at the 6.x→7.x boundary and not since recovered. Since compute now dominates, total solve time regressed ~1.5×.

This is only visible because the measurement splits comm from compute. A blind "solve time" comparison would have wrongly implicated the communication stack.

The plausible fix that isn't: `rocsparse_v2_spmv`

On ROCm 7.x the classic `rocsparse_spmv` is deprecated in favour of `rocsparse_v2_spmv` (a persistent SpMV descriptor + an explicit one-time analysis stage). The obvious hypothesis:

the regression is API/host overhead, and the new API will fix it. The examples let you test that hypothesis directly:

```
make SPMV_V2=1      # compile the rocsparse_v2_spmv path (auto-falls back to v1 on ROCm 6.x)
# compare both paths across the 7.x line:
sbatch --export=ALL,SPMV="v1 v2",VERSIONS="6.4.1 7.0.2 7.2.4 7.13.0",REPEATS=5,METHOD=isend \
    submit_rocm_sweep.sbatch
```

`SPMV_V2=1` also folds in a second micro-optimization applied to both paths: the dense-vector descriptors are created once and rebound with `rocsparse_dnvec_set_values` each iteration instead of being created/destroyed per SpMV.

Result: v2 does not recover the regression

compute (s), min of 5 runs:

ROCm	v1 (rocsparse_spmv)	v2 (rocsparse_v2_spmv)
6.4.1	0.0302 (baseline)	— (v2 not available)
7.0.2	0.0569	0.0547
7.2.4	0.0559	0.0558
7.13.0	0.0567	0.0552

v2 improves 7.x compute by only **0-3 % (within noise)** and leaves the $\sim 1.85\times$ gap vs 6.4.1 intact. Since the per-call descriptor fix is now in the v1 numbers too and v1 is still slow, **the regression is in the rocSPARSE CSR SpMV compute kernel itself**, not in the API surface or host/descriptor overhead. (Migrating to `rocsparse_v2_spmv` is still worthwhile for API longevity — v1 is deprecated — just not as a performance remedy.)

The methodology lesson: **a controlled experiment can disprove a plausible fix**. Because both paths were measured under identical conditions with the comm/compute split, the "it's the deprecated API" hypothesis was cleanly falsified. The next step to localize it is a one-iteration `rocprofv3` kernel trace on 6.4.1 vs 7.2.4 (Chapter 7).

Cross-toolchain: OpenMPI vs cray-mpich

The same solver was also built with **PrgEnv-amd / cray-mpich** (ROCm 7.0.3) and measured identically. `isend`, 4 ranks, same node class & matrix:

stack	ROCm	solve	comm	halo	dot allreduce	compute
OpenMPI 5.0.10	6.4.1	0.051	0.028	0.014	0.014	0.024
OpenMPI 5.0.10	7.2.4	0.111	0.020	—	0.003	0.091

stack	ROCm	solve	comm	halo	dot allreduce	compute
cray-mpich 9.1.0	7.0.3	0.097	0.018	0.015	0.004	0.079

- **The MPI stack is not the differentiator here.** cray-mpich's GPU-Aware communication is on par with (a touch better than) GPU-Aware OpenMPI; the total-solve difference between rows is driven by the **ROCm compute version**, not the MPI implementation. ROCm 7.0.3 lands, as expected, between the 7.0.2 and 7.2.4 compute points.
- **The affinity cliff did not reproduce on Cray** (Chapter 3): an `--exclusive srun` already binds NUMA-locally, so explicit binding bought only ~3–8 %. Same solver, different launcher defaults.

Full details: [STUDY_REPORT.md](#) §5/§5.1 (OpenMPI) and [STUDY_REPORT_PrgEnv-amd.md](#) (cray-mpich).

Takeaways

- Always split comm vs compute before blaming the network — here the network was fine and the library kernel regressed.
- Rebuild per toolchain and compare in one allocation; binaries are not runtime-compatible across major ROCm versions anyway (a 7.x-linked binary against a 6.x runtime fails with an undefined HSA symbol).
- A new API is not automatically a faster API — measure it the same way you measure everything else.

Next: [7. The optimized configuration](#) →

7. The optimized configuration

You've now measured every axis. This chapter assembles them into a recommended setup, a decision checklist, and the directions worth pursuing next.

The optimization path at a glance

Performance progression as each optimization is applied (method = `isend`, 4 ranks, `Dubcova2.pm`, `CG_SEED=12345`, min of runs on one MI300A node):

Step	What changed	Config	Solve (s)	vs prev	vs naive GPU
0	CPU reference (baseline)	CG-CPU, 4 ranks	0.159	—	—
1	Naive GPU: default transport, no affinity	<code>staged</code> , unbound/non-exclusive	9.27	—	1×
2	Fix CPU/GPU affinity (exclusive + <code>gpu_bind.sh</code>)	<code>staged</code> , bound	0.148	63×	63×
3	Switch to GPU-Aware transport	<code>isend</code> , bound	0.051	2.9×	182×
4	Enable SDMA + gang copy engines	<code>isend</code> , bound, <code>HSA_ENABLE_SDMA_GANG=1</code>	0.045	1.13×	206×

The optimized GPU run (0.045 s) is $\sim 206\times$ faster than the naive GPU run and $\sim 3.5\times$ faster than the CPU reference — and **$\sim 90\%$ of that gain is affinity alone** (step 2). Steps 1–3 are ROCm 6.4.1; step 4 is 6.4.3.

Per-axis detail behind each step:

Step 2 — affinity (solve time, bound vs unbound):

method	unbound, not exclusive	--exclusive + <code>gpu_bind.sh</code>	speedup
<code>staged</code>	9.27 s	0.15 s	62×
<code>isend</code>	10.58 s	0.05 s	212×
<code>rccl</code>	7.39 s	0.05 s	148×
<code>alltoallv</code>	8.46 s	0.05 s	169×

Step 3 — transport (bound, ROCm 6.4.1):

method	solve	comm	halo exch	dot allreduce	compute	comm %
<code>staged</code>	0.1477	0.1243	0.1095	0.0148	0.0234	84 %
<code>isend</code>	0.0513	0.0282	0.0142	0.0140	0.0231	55 %

method	solve	comm	halo exch	dot allreduce	compute	comm %
rccl	0.0505	0.0252	0.0160	0.0092	0.0253	50 %
alltoallv_staged	0.0823	0.0560	0.0422	0.0138	0.0263	68 %
alltoallv	0.0512	0.0250	0.0129	0.0121	0.0262	49 %

Step 4 — SDMA vs blit (ROCm 6.4.3, halo-exchange time):

method	blit	sdma	sdma_gang	best Δ vs blit
staged	0.1215	0.1116	0.1129	−8 %
isend	0.0154	0.0119	0.0113	−27 %
rccl	0.0143	0.0129	0.0129	−10 %
alltoallv_staged	0.0480	0.0418	0.0394	−18 %
alltoallv	0.0116	0.0140	0.0121	+21 %

Separate axis — ROCm version / SpMV API (compute time, isend). Not a speedup you "apply" but a version choice: staying on ROCm 6.4.x avoids a ~1.85× compute regression that `rocsparse_v2_spmv` does not recover:

ROCm	v1 rocsparse_spmv	v2 rocsparse_v2_spmv
6.4.1	0.0302 (baseline)	—
7.0.2	0.0569	0.0547
7.2.4	0.0559	0.0558
7.13.0	0.0567	0.0552

Caveats: single node / single matrix / latency-bound regime; the cumulative path mixes ROCm 6.4.1 (steps 2–3) and 6.4.3 (step 4); for the overlap variants the `halo` figure is exposed wait, not raw wire time.

The recommended launch

For a distributed GPU CG solve on MI300A, in priority order:

1. **Exclusive allocation** that owns whole NUMA nodes (`sbatch --exclusive`).
2. **Per-rank GPU + NUMA-local CPU binding** (`gpu_bind.sh` or `set_affinity_mi300a.sh`, launched under `--bind-to none` / `--cpu-bind=none`). This is worth up to ~100× — nothing else comes close.
3. **A GPU-Aware transport** — `isend`, `alltoallv`, or `rccl`. Avoid the `staged` variants unless you have no GPU-Aware MPI; the staging copies cost ~7× the halo time.
4. **SDMA copy engines on** (the default), with `HSA_ENABLE_SDMA_GANG=1` if halo volume is large.
5. **The best available ROCm compute version** — on this workload ROCm 6.4.x compute is ~1.85× faster than 7.x; check whether that still holds on your version before locking it in.


```

cd CG-GPU
make                                     # (or: make SPMV_V2=1 on ROCm 7.x for API longevity)

CG_SEED=12345 \
HSA_ENABLE_SDMA=1 HSA_ENABLE_SDMA_GANG=1 \
mpirun -n 4 --bind-to none ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm isend

```

Reproduce any of the studies end-to-end:

```

sbatch submit_cg_gpu_test.sbatch                # all 5 methods +
convergence + timing
sbatch --export=ALL,REPEATS=5 submit_sdma_sweep.sbatch # SDMA vs blit, all
methods
sbatch --export=ALL,SPMV="v1 v2",REPEATS=5 submit_rocm_sweep.sbatch # ROCm versions x
SPMV API
sbatch --export=ALL,CG_SEED=12345 submit_cg_gpu_cray.sbatch # PrgEnv-amd / cray-mpich

```

Decision checklist: "which transport should I use?"

```

Do you have GPU-Aware MPI?
├ no → staged / alltoallv_staged (host-buffered) are your only correct options.
|     Expect halo to be dominated by D↔H copies. Turn SDMA on.
└ yes → use a GPU-resident transport:
    ├── mostly neighbour exchange, few peers → isend
    ├── dense/irregular all-to-all pattern → alltoallv
    └ want GPU-native, hipGraph-friendly, → rccl
        or plan to overlap on a side stream
(On one node over XGMI these are within noise – pick on ergonomics,
then measure on your real pattern/scale.)

```

Where the time actually goes — and what to optimize next

Once affinity is fixed and a GPU-Aware transport is chosen, the remaining per-iteration cost splits into three buckets. Attack them in ROI order:

A. Dot-product synchronization (highest ROI)

Each iteration does **two** `MPI_Allreduces`, each gated by a full `hipDeviceSynchronize`. That's up to ~27 % of solve on this problem and it is independent of the halo transport, so no amount of transport tuning touches it. Options:

- **Fuse the two dot products** ($r \cdot r$ and $p \cdot Ap$) into a single allreduce where the algorithm allows.

- Adopt a **pipelined / communication-avoiding CG** (Ghysels & Vanroose pipelined CG, or s-step CG) to overlap the global reduction with the SpMV and cut the number of synchronization points.
- Replace `hipDeviceSynchronize` with **stream/event-scoped** sync so unrelated GPU work isn't stalled.

B. Kernel-launch overhead (small per-GPU problems)

At ~16k rows/GPU the loop is launch-latency bound. Capture the per-iteration kernel/BLAS sequence in a **HIP graph**, or fuse the rocBLAS `axpy / scal / dot` chain into custom kernels, to cut launch count.

C. SpMV / comm overlap

- In `rccl`, replace the full `hipDeviceSynchronize` after the gather with an **event + `hipStreamWaitEvent`** so only the RCCL stream waits.
- For the point-to-point variants, split the on-proc SpMV to better hide `MPI_Waitall` latency.

D. The compute-kernel regression (Chapter 6)

If you're on ROCm 7.x and compute-bound, the biggest single factor may be the rocSPARSE CSR SpMV regression. Localize it with a **one-iteration `rocprofv3` kernel trace** on 6.4.1 vs your version to identify the kernel whose duration grew, try `rocsparse_spmv_alg_csr_rowsplit` for this near-uniform-nnz matrix, and file an issue with the reproducer if confirmed.

Analysis directions to deepen the study

1. **Strong & weak scaling** — sweep 1→4→8 GPUs, then multi-node, to expose intra-node (XGMI) vs inter-node behaviour, where `rccl` and GPU-Aware `alltoallv` should start to diverge.
2. **Problem-size sweep** — larger matrices per GPU shift the balance from latency-bound to bandwidth/compute bound and shrink the exposed halo of the overlap variants; report a compute-bound regime to complement this latency-bound one.
3. **Raw vs exposed communication** — add non-overlapped timing or profiler traces to separate wire time from exposed wait for the overlap variants.
4. **Profiler-level attribution** — `rocprofv3` / `rocprofiler-systems` to attribute time to SpMV, dot, allreduce, and copies, and to confirm the NUMA-locality benefit at the hardware-counter level.
5. **Communication-volume model** — correlate measured halo time with actual per-rank ghost byte counts (from `form_comm`) to build a latency + bandwidth model and predict behaviour at scale.

The one-paragraph summary

The path to an optimized, comparable performance measurement on AMD multi-GPU nodes is: **bind first** (affinity dominates everything), **fix the seed and take warm minimums** (so runs are comparable), **split comm from compute** (so you optimize the right thing), then compare **transport** (use GPU-Aware MPI; the GPU-resident variants tie on one node), **copy engine** (SDMA on, gang for big halos), and **toolchain/version** (watch for compute-side regressions the network numbers won't reveal). The examples in this repo are instrumented and scripted so that each of these is a controlled, reproducible experiment rather than a guess.

← Back to the [tutorial index](#) · In-depth data: [STUDY_REPORT.md](#) · [STUDY_REPORT_PrgEnv-amd.md](#)

8. Profiling: measuring communication and compute

The solver already reports a coarse **communication vs. compute** split from `MPI_Wtime` timers (`g_halo_time`, `g_allreduce_time` in `cg.cpp`; see the `comm total / halo exchange / dot allreduce / compute (rest)` lines). This chapter shows how to go deeper with the profilers installed on the system — attributing time to individual **kernels**, **transports** (SDMA/blit copies, RCCL, MPI point-to-point/collective), and **hardware limits** (HBM/DRAM bandwidth, FLOP/s) — for **both** the CPU reference and the GPU solver.

Which profiler for which job

Profiler	Load	Scope	Measures here
rocprofv3	<code>module load rocm</code>	GPU (CG-GPU)	Per-kernel time (compute); memory-copy, HIP, and RCCL API traces (communication); ATT instruction-level trace (<code>--att</code> , §A.4)
rocprof-compute	<code>module load rocm</code>	GPU	Roofline: achieved HBM bandwidth vs peak → proves SpMV is memory-bound
rocprofiler-systems	<code>module load rocprofiler-systems</code>	GPU + host	Perfetto timeline : halo-exchange overlap, SDMA vs blit copies, RCCL/MPI on the host thread
TAU	<code>module load tau</code>	MPI + GPU	Per-call MPI time (<code>MPI_Allreduce</code> , <code>MPI_Isend</code> , <code>MPI_Alltoallv</code>) + communication matrix + GPU kernels
HPCToolkit	<code>module load hpctoolkit</code>	MPI + GPU	Call-path sampling: MPI wait time vs compute, GPU kernel attribution
likwid	<code>module load likwid</code>	CPU (CG-CPU)	DRAM bandwidth (<code>MEM_DP</code>) and <code>FLOPS_DP</code> → CPU roofline of the reference solver
AMD uProf	<code>add .../AMDuProf_5.3-518/bin to PATH</code>	CPU (CG-CPU)	CPU hotspots (SpMV loop, dot) and memory/bandwidth analysis
Linux perf	none (in-box; run in an allocation)	CPU (CG-CPU)	Baseline HW counters: IPC + cache-miss rate (<code>perf stat</code>); cycle hotspots (<code>perf record/report</code>) (§K)
cachegrind (Valgrind)	<code>/usr/bin/valgrind</code> (compute node)	CPU (CG-CPU)	Deterministic simulated cache model: D1/LL miss counts per function/line (§L)
IntelliKit	<code>module load rocm intellikit</code>	GPU (CG-GPU)	Decoded metrics (<code>metrix</code>), kernel isolation (<code>kerncap</code>), correctness validation (<code>accordo</code>), source-level SQT (<code>linex</code>) — agent/MCP-driven (§H)

Profiler	Load	Scope	Measures here
roofline-extractor	<code>module load rocm</code> <code>roofline-extractor</code>	GPU (CG-GPU)	Automated per-kernel percent-of-peak roofline (AI @ HBM/L2/L1/LDS, limiter, % of roof, HTML plot, cross-arch projection) (§I)
rocBudAI	<code>salloc --</code> <code>comment=ollama;</code> <code>module load rocbudai</code>	GPU + host	AI assistant that drives the whole stack (build→profile→analyse→optimise loop, airgapped, writes <code>report.md</code>) (§J)

The GPU-native tools (`rocprofv3`, `rocprof-compute`, `rocprofiler-systems`, `roofline-extractor`, `IntelliKit`) target `CG-GPU`; the CPU tools (Linux `perf`, `Valgrind` `cachegrind`, `uProf`; `likwid` where supported) target `CG-CPU`. **TAU** and **HPCToolkit** are whole-application and work on either (they are the best way to see the MPI communication that the ROCm GPU tools do not trace). **rocBudAI** (§J) can drive any of them for you.

Verified on AAC6 / MI300A (ROCm 6.4.3, OpenMPI 5.0.10, 4 ranks, Dubcova2.pm). All commands below were run against these solvers. Confirmed working: `rocprofv3` (kernel + sys trace), `rocprof-compute` (roofline + Top-Kernels — SpMV `rocsparse::csrsvn` and the `rocBLAS axpy / dot / scal` kernels), `rocprofiler-systems` (Perfetto trace), **TAU** (MPI breakdown via `pprof`), **HPCToolkit** (`hpcrun` → `hpcstruct` → `hpcprof` database), and AMD `uProf` (hotspots — top is the CSR SpMV). **Exception:** `likwid-perfctr` counters are **not supported** on the MI300A APU CPU in this build (see §F).

Ground rules (or your numbers are noise)

Everything from [chapter 3](#) still applies under a profiler — more so, because profiling adds overhead:

- Profile **inside an affinity-bound**, `--exclusive allocation` using `gpu_bind.sh` / `set_affinity_mi300a.sh`. Bad affinity dominates everything.
- **Fix the RHS** with `CG_SEED` so every profiled run solves the same system.
- Profile **one transport at a time** (`staged`, `isend`, `rccl`, `alltoallv`, `alltoallv_staged`) and compare across runs.
- Give **each rank its own output directory** (use `$OMPI_COMM_WORLD_RANK`).

Launch note (verified on AAC6 / MI300A, ROCm 6.4.3, OpenMPI 5.0.10).

Use an `sbatch` job (or `salloc`) that requests GPUs, e.g. `sbatch -N1 --exclusive --gpus=4`. An `sbatch` script body runs on the compute node, which matters for the **CPU counter tools** (`uProf`) — they must execute where the ranks run, so launch them **per rank under `mpirun`**, not by wrapping `mpirun` itself. If `mpirun` complains about slots under a one-task step, add `--oversubscribe` (or `export OMPI_MCA_rmaps_base_oversubscribe=true`). The GPU-native tools (`rocprofv3/rocprof-compute/rocprofiler-systems`) run fine under `mpirun` and write per-rank output.

Tip — label the timeline. For sharper GPU traces, wrap the halo and all-reduce regions in ROCTx ranges (`#include <roctracer/roctx.h>; roctxRangePush("halo") / roctxRangePop()`), then link `-lroctx64`. `rocprofv3` and `rocprofiler-systems` will show these named ranges next to the kernels.

A. CG-GPU — rocprofv3 (kernels + transports)

A.1 Compute: per-kernel trace

```
module load rocm openmpi
cd CG-GPU && make
CG_SEED=12345 mpirun -n 4 ./gpu_bind.sh \
  rocprofv3 --kernel-trace --output-format csv \
  -d prof_kern_rank_`${OMPI_COMM_WORLD_RANK:-0}` \
  -- ./cg_gpu src/Dubcova2.pm rccl
```

The kernel CSV shows where GPU compute goes: `rocsparse_spmv` (dominant), the `rocsparse_dgthr` gather that packs the send buffer, and the rocBLAS `ddot` / `daxpy` / `dscal` kernels. Roll up total ns per kernel to rank the compute.

A.2 Communication: memory-copy + RCCL/HIP traces

```
# staged / alltoallv_staged: see the D->H and H->D staging copies
CG_SEED=12345 mpirun -n 4 ./gpu_bind.sh \
  rocprofv3 --memory-copy-trace --hip-trace --output-format pfttrace \
  -d prof_comm_rank_`${OMPI_COMM_WORLD_RANK:-0}` \
  -- ./cg_gpu src/Dubcova2.pm staged

# rccl: --sys-trace additionally captures the RCCL API (ncclSend/ncclRecv)
CG_SEED=12345 mpirun -n 4 ./gpu_bind.sh \
  rocprofv3 --sys-trace --output-format pfttrace \
  -d prof_rccl_rank_`${OMPI_COMM_WORLD_RANK:-0}` \
  -- ./cg_gpu src/Dubcova2.pm rccl
```

Open the per-rank `*.pfttrace` at <https://ui.perfetto.dev>. For `staged` you see the D↔H staging copies around each SpMV; for `rccl` you see `ncclSend/ncclRecv` on the `rccl_stream` overlapping the on-proc SpMV on the default stream. This is the halo-exchange cost the solver rolls into `g_halo_time`, now resolved by transport. (rocprofv3 traces RCCL and HIP but **not** MPI itself — use TAU or HPCToolkit in sections D/E for the `MPI_Isend` / `MPI_Alltoallv` / `MPI_Allreduce` timing.)

A.3 Bandwidth counters (optional)

```
printf 'pmc: FETCH_SIZE WRITE_SIZE L2CacheHit VALUeBusy\n' > counters.txt
mpirun -n 4 ./gpu_bind.sh rocprofv3 -i counters.txt --output-format csv \
  -d prof_pmc_rank_`${OMPI_COMM_WORLD_RANK:-0}` -- ./cg_gpu src/Dubcova2.pm rccl
```

`FETCH_SIZE` + `WRITE_SIZE` ÷ kernel time = achieved HBM bandwidth per kernel — or let `rocprof-compute` do the math (next section).

A.4 Instruction-level: Advanced Thread Trace (ATT)

The traces above stop at the kernel boundary. **Advanced Thread Trace** (ATT, `rocprofv3 --att`) goes one level deeper: it records wavefront execution at the **ISA-instruction** granularity on selected compute units, so you get a per-instruction hotspot/hit map, stall reasons, VALU/VMEM issue behaviour, and occupancy for a single kernel — the tool for asking why the SpMV is bandwidth-bound (memory-stall dominated) rather than just that it is.

ATT produces an enormous amount of data, so you **must** target it:

- **One kernel** — filter with `--kernel-include-regex` (here the `rocsparse` SpMV, e.g. `csmv`), optionally `--att-consecutive-kernels N` to bound how many invocations are captured.
- **One rank** — run `-n 1`; the compute per rank is representative.
- **A few CUs / SIMDs** — `--att-target-cu` (default 1) and `--att-simd-select` (default 0xF), plus `--att-shader-engine-mask` (default 0x1).

MI300A is **gfx942 (gfx9)**, so the gfx9-only options apply: `--att-perfcounters` (a counter list sampled alongside the trace) and `--att-activity 8` (HW activity counters; AMD's recommended period).

```
module load rocm/7.13.0 openmpi # 7.13.0 (or 7.12.0) ships the ATT decoder
cd CG-GPU && make
# Instruction trace of the SpMV kernel on CU 1 of one rank:
CG_SEED=12345 mpirun -n 1 --oversubscribe ./gpu_bind.sh \
rocprofv3 --att \
  --att-library-path $ROCM_PATH/lib \
  --att-target-cu 1 \
  --att-shader-engine-mask 0x1 \
  --att-simd-select 0xF \
  --att-activity 8 \
  --att-consecutive-kernels 1 \
  --kernel-include-regex 'csmv' \
  -d att_spmv \
  -- ./cg_gpu src/Dubcova2.pm rccl
```

The decoded output lands under the `-d` directory: `*_gfx942_code_object_id*.out` files hold the **ISA disassembly annotated with per-instruction hit counts**, and a `ui_output_agent*_dispatch_*` folder holds per-wavefront JSON for the ROCm ATT viewer (a `*_results.db` rocprof SQLite database is written too). Read it to confirm the SpMV spends its cycles on `global_load/flat_load` (VMEM) waits rather than in VALU — the instruction-level restatement of "CG is memory-bound".

Decoder library — use ROCm ≥ 7.12 (measured). ATT collection is built into `rocprofv3`, but decoding needs the separate closed-source `rocprof-trace-decoder` library. On this cluster it ships with `rocm/7.12.0` and `rocm/7.13.0` (`$ROCM_PATH/lib/librocprof-trace-decoder.so`) but **not** with `rocm/7.2.4` or earlier — there the run aborts with `Fatal error: rocprof-trace-decoder library path not found`. So load a

7.12+ `rocm` module and point `--att-library-path` at `$ROCM_PATH/lib` (or install the package / grab a release from <https://github.com/ROCm/rocmprof-trace-decoder> and point at that). **Verified decode-to-ISA on MI300A with `rocm/7.13.0`** (decoder `0.1.7`). The other `rocmprofv3` modes (A.1–A.3) need no decoder.

B. CG-GPU — `rocmprof-compute` (roofline)

The compute per rank is representative, so characterize the kernels on a single rank:

```
module load rocm
rocmprof-compute profile -n cg_spmv -- ./cg_gpu src/Dubcova2.pm rccl 12345
rocmprof-compute analyze -p workloads/cg_spmv/MI300* | less
```

Read the **roofline** and **memory chart**: the SpMV and rocBLAS kernels sit on the **HBM-bandwidth roof** (low arithmetic intensity) — CG is memory-bound, so the achieved GB/s vs. peak is the practical compute ceiling. `--roof-only` is a faster, fewer-pass variant.

C. CG-GPU — `rocprofiler-systems` (timeline of overlap)

```
module load rocprofiler-systems
CG_SEED=12345 mpirun -n 4 ./gpu_bind.sh \
  rocprof-sys-run -- ./cg_gpu src/Dubcova2.pm rccl
```

This produces a per-rank Perfetto trace. Line up the GPU-kernel row, the memory-copy row, and the host row to see:

- the **on-proc SpMV overlapping** the in-flight halo messages (the "exposed wait" the README mentions for `isend/rccl/alltoallv`),
 - **SDMA vs blit** copies for GPU-Aware transports — rerun with `HSA_ENABLE_SDMA=0` (blit) vs `=1` (SDMA) and compare the copy rows, tying the timeline back to the [SDMA vs blit chapter](#).
-

D. CG-GPU / CG-CPU — TAU (the MPI communication view)

TAU intercepts MPI (and, with `-rocm`, ROCm) so it directly measures the communication the ROCm tools miss:

```
module load tau rocm openmpi
# GPU solver: MPI + ROCm
CG_SEED=12345 mpirun -n 4 ./gpu_bind.sh \
  tau_exec -T MPI,ROCM -rocm ./cg_gpu src/Dubcova2.pm rccl
# CPU reference: MPI only
```



```
mpirun -n 4 tau_exec -T MPI ./cg_cpu src/Dubcova2.pm
```

```
pprof          # text summary, or:
paraprof       # GUI: per-call MPI time + communication matrix
```

TAU reports time in `MPI_Allreduce` (the dot-product reduction → `g_allreduce_time`), `MPI_Isend/MPI_Irecv/MPI_Waitall` or `MPI_Alltoallv` (the halo exchange → `g_halo_time`), plus GPU kernel time — the same split the solver prints, now attributed per MPI call and shown as a **communication matrix** across ranks. (For this small test matrix `MPI_Init` dominates the profile; use a larger matrix for a meaningful loop breakdown. TAU may return a nonzero exit at teardown after ROCm tracking — the `profile.*` files are still written and readable by `pprof/paraprof`.)

E. CG-GPU / CG-CPU — HPCToolkit (call-path sampling)

```
module load hpctoolkit rocm openmpi
# GPU solver: CPU call paths + AMD GPU operations
CG_SEED=12345 mpirun -n 4 ./gpu_bind.sh \
  hpcrun -e CPUTIME -e gpu=amd -o cg_gpu.m ./cg_gpu src/Dubcova2.pm rccl
# CPU reference:
mpirun -n 4 hpcrun -e CPUTIME -o cg_cpu.m ./cg_cpu src/Dubcova2.pm

hpcstruct ./cg_gpu          # recover program structure
hpcprof -S cg_gpu.hpcstruct cg_gpu.m -o cg_gpu.d
hpcviewer cg_gpu.d         # call-path profile; hpctraceviewer for timeline
```

HPCToolkit shows how much time each rank spends in MPI wait vs. compute vs. GPU kernels along the full call path — ideal for spotting a load-imbalanced halo exchange or a rank stalling in `MPI_Allreduce`. (Run `hpcstruct/hpcprof` on the compute node so their runtime libs are present; if `hpcprof` warns about a partial struct-file match, pass the suggested `-R '<build>'='<build>/.'` remap.)

F. CG-CPU — likwid (CPU roofline)

On a supported CPU, likwid gives hardware-counter bandwidth/FLOP/s per rank:

```
module load likwid openmpi
cd CG-CPU && make
likwid-mpirun -mpi openmpi -np 4 -g MEM_DP ./cg_cpu src/Dubcova2.pm # DRAM bandwidth
likwid-mpirun -mpi openmpi -np 4 -g FLOPS_DP ./cg_cpu src/Dubcova2.pm # DP MFLOP/s
```

Compare the summed DRAM bandwidth to the node's STREAM peak: a memory-bound CG should land near it. To isolate the SpMV from the dot products, add the likwid **Marker API** (`likwid_markerStartRegion("spmv") / ...StopRegion`) and build with `-DLIKWID_PERFMON -llikwid`.

⚠ **MI300A caveat (measured).** On the MI300A APU node, `likwid-perfctr` (v5.5.1) reports "Unsupported AMD Zen Processor / K19" and cannot initialize its event groups — the integrated Zen4 CPU is not in this build's perfmon map, so `MEM_DP / FLOPS_DP` are unavailable there. `likwid-topology` still works on supported hosts. **For CPU counters/hotspots on MI300A, use Linux `perf` (§K), AMD uProf (§G), or `rocprow-compute` for the GPU roofline instead.** `likwid` remains useful on conventional EPYC login/compute nodes.

G. CG-CPU — AMD uProf

uProf **does** work on the MI300A node. Profile it **per rank under `mpirun`** (so uProf runs on the compute node where the ranks execute), each writing to its own directory, then generate the report from that **directory**:

```
export PATH=$PATH:/nfsapps/ubuntu-24.04-nightlies/opt/AMDuProf_5.3-518/bin
cd CG-CPU
# Time-based (hotspot) profiling, one uProf per rank:
mpirun --oversubscribe -n 4 bash -c \
    'AMDuProfCLI collect --config tbp -o uprof_r${OMPI_COMM_WORLD_RANK} ./cg_cpu src/
    Dubcova2.pm'
# Report from the collection DIRECTORY (not the session.uprof file):
AMDuProfCLI report -i uprof_r0          # writes uprof_r0/report.csv
```

Measured on MI300A, `report.csv` lists the hottest functions — for this solver the top entry is the CSR SpMV (`spmv(double, ParMat&, ...)`), then `main` and the `std::map` column-index lookups. The memory/bandwidth analysis config (see `AMDuProfCLI collect --help`) adds DRAM traffic and cache behavior — the CPU-side analog of the `rocprow-compute` roofline. (Requires `perf_event_paranoid` low enough to sample; the MI300A compute nodes are configured permissively.)

H. CG-GPU — IntelliKit (agent-first: metrix / kerncap / accordo / linex)

IntelliKit (module `intellikit/main`, loaded **after** a `rocm` module) is an "agent-first" ROCm profiling and validation toolkit built on `rocprofiler-sdk`. Where the tools above emit raw counters and traces, IntelliKit gives **decoded, human-readable** metrics and **kernel isolation/validation** — and every tool also ships as an MCP server so an AI agent can drive it. Installed tools: `metrix`, `kerncap`, `accordo` (CLIs); `linex`, `nexus` (SQTT / HSA, MCP-driven); `rocm_mcp`, `uprof_mcp` (MCP servers).

Invocation note (this build). Only `accordo` is installed as a `bin` script. Invoke the others through the module's Python entrypoints — e.g. define shell wrappers once per session:

```
```bash module load rocm/7.2.4 intellikit/main kerncap(){ python -c "from
kerncap.cli import main; main()" "$@"; } metrix(){ python -c "from metrix.cli.main
import main; main()" "$@"; }
```

## accordo is on PATH directly

---

```

H.1 metrix — GPU profiling, decoded

`metrix` collects `rocprofiler-sdk` counters and reports them as named metrics (bandwidths, cache hit rates, arithmetic intensity) instead of raw PMC — the low-friction way to answer "is the SpMV bandwidth-bound?".

```
# Decoded HBM/L2 bandwidth for just the SpMV kernel, top 5 dispatches
metrix profile -p memory_bandwidth -k 'csrnmv' --top 5 \
    "./cg_gpu src/Dubcova2.pm rccl 12345"
# profiles: quick | memory | memory_bandwidth | memory_cache | compute
# or a custom set: metrix profile -m memory.hbm_bandwidth,memory.l2_hit_rate ...
```

Verified on MI300A (ROCm 7.2.4) against the `rocparse::csrnmv` SpMV:

```
Dispatch #1420: csrnmv_general_kernel    Duration avg=8.84 µs
HBM Bandwidth Utilization      4.75 Percent
HBM Read Bandwidth             245.55 GB/s
L2 Cache Bandwidth Util.       668.79 GB/s
```

Reading it: for this small test matrix the SpMV is largely **L2-resident** — L2 bandwidth (~669 GB/s) far exceeds HBM (~245 GB/s) and HBM utilization is only ~4.7%. Scale the matrix up and the working set spills to HBM, pushing the kernel onto the HBM roof — the decoded restatement of the [rocprof-compute roofline](#) in §B. `metrix` and `rocprof-compute` agree; `metrix` is faster to read, `rocprof-compute` plots the roofline.

H.2 kerncap — isolate the SpMV kernel

`kerncap` ranks kernels and, more usefully, **extracts a single kernel into a standalone reproducer** you can iterate on without the full solver:

```
kerncap profile -- ./cg_gpu src/Dubcova2.pm rccl 12345      # rank kernels by time
kerncap extract csrnmv_general --cmd './cg_gpu src/Dubcova2.pm rccl 12345' \
    --language hip -o spmv_repro/                          # standalone reproducer
kerncap replay spmv_repro/                                 # VA-faithful re-dispatch
kerncap validate spmv_repro/                               # outputs match capture?
```

Verified `kerncap profile` on MI300A ranks the CG kernels by time — the `rocparse::csrnmv` SpMV plus the rocBLAS `axpy / dot / scal` and the rocclr fill/copy buffers — the same compute breakdown as §A.1, produced in one command.

H.3 accordo — correctness validation of an optimization

When you optimize the SpMV, `accordo` intercepts a kernel and compares outputs between a **reference** and an **optimized** binary within tolerance:

```
accordo validate --kernel-name csrmln_general_kernel \
  --ref-binary ./cg_gpu_ref --opt-binary ./cg_gpu_opt \
  --atol 1e-8 --rtol 1e-5
```

This is the guard-rail for the optimization path in [chapter 7](#): prove a faster kernel still computes the same result before trusting its speedup.

H.4 linex / nexus and the MCP servers

- **linex** — source-level SQT: maps GPU cycles back to your **source lines**. It is the **decoded companion to the raw ATT/SQT trace in §A.4** — same underlying thread trace, presented against source. (SQT decoding needs the same `rocpf-trace-decoder` library noted in §A.4.)
- **nexus** — HSA packet / kernel source extractor (research tool for intercepting and analysing dispatched kernels).
- **MCP servers** — `linex-mcp`, `nexus-mcp`, `kerncap-mcp`, `metrix-mcp`, `accordo-mcp`, plus `rocm_mcp` (rocminfo/amd-smi/HIP compiler & docs) and `uprof_mcp` (AMD uProf). These expose the tools to an MCP client so an agent can profile, extract, and validate CG kernels conversationally.

Verified on PPAC / MI300A (gfx942, ROCm 7.2.4, intellikit/main). `metrix` profile (memory_bandwidth on the SpMV) and `kerncap` profile/extract run against `cg_gpu` and use `rocpf-trace-decoder` counters directly (no extra decoder). `linex` / `nexus` are SQT/HSA and MCP-server-first; `linex` source mapping needs the `rocpf-trace-decoder` library (see §A.4).

I. CG-GPU — roofline-extractor (percent-of-peak roofline)

`roofline-extractor` (module `roofline-extractor`, load after `rocm`) is an automated **percent-of-peak** roofline analyser. It drives `rocpf-v3` for you (four counter passes + one kernel-trace pass), then reports, per kernel, arithmetic intensity at each memory level (HBM/L2/L1/LDS), the active performance limiter, achieved-vs-peak throughput, and % of roofline reached — with an optional interactive HTML plot and cross-architecture projection (`--proj-arch`). It is the turnkey complement to [rocpf-compute \(§B\)](#): less to drive, and it prints the headline number directly ("you are at X% of the HBM roof").

```
module load rocm
module load roofline-extractor
# End-to-end: profiles cg_gpu and prints the per-kernel roofline breakdown.
roofline-extractor-profile -o rex_out -- ./cg_gpu src/Dubcova2.pm rccl 12345
# add --proj-arch MI355X to also project runtime/speedup onto another GPU
```

Verified on MI300A (gfx942, ROCm 7.13.0) against `cg_gpu`:

```
<rocBLAS kernel> contribution 10.7% GPU time
  Arithmetic intensity (HBM): 0.5752    limiter: HBM_BW (MI300A)
  Achieved 129.4 GFLOPS/s vs 2121 peak → 6.1% of linear roofline
  ...
Average percent of linear roofline achieved: 17.2 %
Total application GPU time on MI300A: 7.48e6 ns
```

The low arithmetic intensity (~0.58 FLOP/byte) and the `HBM_BW` limiter are the percent-of-peak restatement of the memory-bound result from §B / §H.1: CG lives on the bandwidth roof. Outputs (`counters.csv` , `*_EXTRACTED*.csv` , `counters.html`) land in `rex_out/` ; open the HTML to see each kernel plotted on the roofline. For a multi-rank/phase view, collect per-run bundles and use `roofline-extractor-extract -D <bundle_dir>`. See `man roofline-extractor` and the `METRICS_SUMMARY.md` / `METRICS_DETAILED.md` under the install root for metric formulas and per-architecture peaks.

Arch note. The tool needs an MI-series arch it recognizes (MI250/MI300A/ MI300X/ MI325X/MI350X/...). On a **CPX**-mode node the virtual GPUs may not match a known arch string — run on **SPX**, or pin `--arch MI300A`.

J. CG-GPU — rocBudAI (AI profiling assistant)

`rocBudAI` (module `rocbudai`) is an AI performance-engineering assistant that **drives the whole stack above for you**. It wires the OpenCode TUI to a locally-hosted model (Ollama) pre-seeded with the AMD profiling toolchain (`rocprofv3`, `rocprof-compute`, `rocprof-sys`, `rocpd`, `rocm-smi`), then runs a build → profile → analyse → optimise → re-profile loop, writing a running `report.md` whose claims are tagged `[FACT]` / `[INFERENCE]` / `[OPINION]`.

It runs **only on a Slurm-allocated compute node**, and the `--comment=ollama` allocation is air-gapped (source, prompts, and model output stay on the node):

```
# 1. Allocate an SPX MI300A node with the local model daemon started for you:
salloc -p PPAC_MI300A_SPX --exclusive --comment=ollama --time=02:00:00
# 2. Point it at this example:
cd ~/HPCTrainingExamples/MPI-examples/cg-solver-example/CG-GPU
# 3. Launch the assistant (auto-starts the TUI, pre-configured for AMD profiling):
module load rocbudai
```

A seven-question interview (app type, ROCm version, modules, build command, run command, figure of merit) configures the run. For this example answer roughly: HIP/C++ + MPI; rocm/7.13.0 + openmpi; build `make`; run `mpirun -n 4 ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm rccl`; FoM = CG solve time (or % of roofline). The agent then profiles with the tools in this chapter and proposes optimizations. Use it when you want the workflow orchestrated and explained rather than run by hand — but still check its conclusions against the raw tool output above. See `man rocbudai` for airgap enforcement, session

management, and ASK vs AUTO-RUN permission modes; verify the sandbox with `rocbudai-airgap-check`.

K. CG-CPU — Linux perf (always-available CPU baseline)

`perf` needs no modules and no special build — the zero-friction CPU baseline, and the natural fallback where likwid's counters are unsupported on MI300A (§F). The MI300A **compute** nodes run with `perf_event_paranoid = -1`, so hardware counters are fully available (the **login** node is restricted — run `perf` inside an allocation).

```
cd CG-CPU && make CXXFLAGS="-O3 -g -std=c++17" # -g for symbol resolution
# Per-rank counter summary (IPC + cache behaviour), one file per rank:
mpirun -n 4 bash -c 'perf stat -o perf_r${OMPI_COMM_WORLD_RANK}.txt \
  -e cycles,instructions,cache-references,cache-misses,L1-dcache-loads,L1-dcache-load-misses \
  ./cg_cpu src/Dubcova2.pm 12345'
# Hotspot sampling → where the cycles go:
mpirun -n 1 perf record -g -o perf.data ./cg_cpu src/Dubcova2.pm 12345
perf report -i perf.data # interactive; or `perf annotate` for source+asm
```

Verified single-rank on MI300A (`perf stat`):

| | | |
|-------------|-----------------------|-----------------------------------|
| 726,966,836 | cycles | |
| 602,835,327 | instructions | # 0.83 insn per cycle |
| 18,270,624 | cache-references | |
| 2,410,388 | cache-misses | # 13.19% of all cache refs |
| 205,921,430 | L1-dcache-loads | |
| 4,193,649 | L1-dcache-load-misses | # 2.04% of all L1-dcache accesses |

The low IPC (0.83) and the non-trivial last-level cache-miss rate (~13%) are the CPU signature of the memory-bound SpMV — the CPU analog of the GPU HBM-bound roofline (§B/§I). `perf record`/`perf report` then attribute cycles to the SpMV row-loop and the CSR index gather. (`perf list | grep -i cache` shows the memory events your CPU exposes; wrap `perf stat` under `mpirun` for per-rank files as above rather than measuring the launcher.)

L. CG-CPU — Valgrind cachegrind (deterministic cache model)

Where `perf` reads noisy hardware counters, **cachegrind** simulates the cache hierarchy, so its miss counts are **deterministic and repeatable** — ideal for comparing two SpMV implementations without run-to-run counter jitter. It is a CPU binary-instrumentation tool from the Valgrind suite; `valgrind` is installed on the MI300A **compute** nodes (`/usr/bin/valgrind`; not on the login node). Expect a ~20–50× slowdown, so use a fixed, small matrix.

```
cd CG-CPU && make CXXFLAGS="-O2 -g -std=c++17" # -g (and -O2) for clean annotation
valgrind --tool=cachegrind --cache-sim=yes \
  --cachegrind-out-file=cg.cachegrind.out \
  ./cg_cpu src/Dubcova2.pm 12345
cg_annotate cg.cachegrind.out # file:function miss breakdown
cg_annotate cg.cachegrind.out src/cg.cpp # annotate a source file line-by-line
```

Verified on MI300A:

```
D refs:      46,177,102 (32,130,783 rd + 14,046,319 wr)
D1 misses:   653,773      D1 miss rate: 1.4%
LLd misses:  136,346      LLd miss rate: 0.3%
```

`cg_annotate` breaks the D1/LL misses down **per function and per source line**, so you can see the misses land in the CSR SpMV row-loop and the column-index-driven gather of `x[col[j]]` — the classic irregular-access hotspot of sparse mat-vec. (Because CG reads a gzipped matrix, `gzgets`/decompression can dominate a tiny run; fix the seed and use a larger matrix, or focus on the solve-loop functions, for a representative picture. `--branch-sim=yes` adds branch-misprediction stats.)

M. perf_events security & access control

`perf` (§K) reads hardware performance counters, execution-context registers, and sampled addresses — data that can leak sensitive information from other processes. The kernel therefore gates `perf_events` behind an access-control model you must understand to (a) know why a command is denied and (b) request the right, least-privilege fix from your admins. This section summarises the [kernel perf-security guide](#) as it applies to the CG example; the runnable demo is [CG-CPU/perf_security_demo.sh](#) and the full implementation/demo plan is [chapter 9](#).

The `perf_event_paranoid` scope ladder

An unprivileged user's scope is set by `/proc/sys/kernel/perf_event_paranoid`:

| Value | What an unprivileged user may do |
|----------------|--|
| -1 | Everything; per-cpu <code>perf_event_mlock_kb</code> limit ignored (least secure) |
| >= 0 | Per-process and system-wide (<code>-a</code>); excludes raw/ftrace tracepoints |
| >= 1 | Per-process only (no system-wide); user + kernel events |
| >= 2 | Per-process, user-space events only (no kernel-space sampling) |

On this cluster the **exclusive compute nodes run** `paranoid = -1` (full access — verified), while the **login node is restricted** (`paranoid = 4`, i.e. `> 2`). That is why the rule throughout this chapter is profile inside an allocation. `perf stat/perf record` on your own

`cg_cpu` (per-process) work at any level `<= 2`; `perf stat -a` (system-wide, e.g. uncore/IMC memory-bandwidth counters) needs `<= 0`.

CAP_PERFMON — the least-privilege alternative

Lowering `paranoid` system-wide is blunt. The secure alternative is the `CAP_PERFMON` capability (Linux ≥ 5.9), which grants `perf` scope without full root and is preferred over the legacy `CAP_SYS_ADMIN`. Admins typically gate it to a group and stamp it on the `perf` binary:

```
groupadd perf_users
chgrp perf_users "$(command -v perf)" && chmod o-rwx "$(command -v perf)"
setcap "cap_perfmon,cap_sys_ptrace,cap_syslog=ep" "$(command -v perf)"
getcap "$(command -v perf)" # cap_sys_ptrace,cap_syslog,cap_perfmon+ep
```

Members of `perf_users` then profile at any `paranoid` level. (`cap_syslog` allows resolving kernel symbols via `/proc/kallsyms`; `cap_sys_ptrace` is not needed on ≥ 5.9 but is harmless; add `cap_ipc_lock` for `perf top`.) Where `setcap` is not possible (e.g. `nosuid` filesystem) the guide's `capsh/sudo` "privileged `perf` shell" achieves the same via the ambient capability set.

The [demo](#) automates this as a **least-privilege overlay on one ROCm release**:

[perf_users_setup.sh](#) creates the group, adds users, and gates (`chgrp + chmod o-rwx`) + `setcaps` the profilers of a single release, while **leaving `perf_event_paranoid` open** so profilers from other releases keep working unchanged. A real-world wrinkle it handles: the local `perf` ELF is on **xfs** (`setcap` works), but ROCm is on **NFS**, which cannot hold file-capability xattrs — there group members get `CAP_PERFMON` via the `capsh` privileged shell instead. (The script uses the lean `cap_perfmon,cap_syslog=ep`; `cap_sys_ptrace` from the guide's example is dropped as unneeded on ≥ 5.9 .) To **prove** the overlay, [perf_paranoid_test.sh](#) (run once under `sudo` in an exclusive allocation) raises `perf_event_paranoid` and shows an ordinary `perf_users` member still lands a system-wide `perf stat -a` via `CAP_PERFMON`, then restores the original value.

Resource limits that bite multi-rank runs

Two per-process resource limits (not security scope, but they cause the same "cannot proceed" symptom) matter when profiling all MPI ranks at once:

- `perf_event_mlock_kb` (here 516 KiB, per cpu) — the ring-buffer budget. The first `perf` process can grab it all and starve the other ranks. Cap each rank's buffer with `perf record --mmap-pages=N` (or `perf top -m N`).
- `RLIMIT_NOFILE` (`ulimit -n`) — `perf` opens $\geq \text{events} \times \text{cpus}$ file descriptors; large event lists on many cores can exhaust it.

`CAP_IPC_LOCK` lifts the `mlock` limit for privileged `perf` users.

Why this does not block on the ROCm version

`perf_events` security is a **kernel** feature — orthogonal to ROCm. The [demo](#) therefore pins a **single** `rocm` module (only to build `cg_cpu`) and does **not** sweep ROCm versions: the

paranoid ladder, `CAP_PERFMON`, and the resource limits behave identically under every ROCm module on a given kernel. Better still, because the setup **leaves** `perf_event_paranoid open`, you can apply the `perf_users / CAP_PERFMON` overlay to **one release's** profilers and every other release keeps working untouched — a safe, one-at-a-time rollout rather than a global change. (Contrast the GPU ATT decoder in §A.4, which genuinely needs `rocm >= 7.12.`)

Mapping tools to the tutorial's questions

| Question | GPU (CG-GPU) | CPU (CG-CPU) |
|---|--|--|
| Which transport is fastest for the halo exchange? | <code>rocprofv3 --sys-trace + rocprofiler-systems timeline</code> ; TAU MPI matrix | TAU / HPCToolkit MPI time |
| How costly is the dot-product all-reduce? | TAU / HPCToolkit <code>MPI_Allreduce</code> time (→ <code>g_allreduce_time</code>) | same |
| Is SpMV hitting the bandwidth ceiling? | <code>rocprof-compute</code> roofline; roofline-extractor % of peak (§I); IntelliKit <code>metrix</code> (decoded, §H.1) | perf cache-miss rate + IPC (§K); uProf memory (likwid perfmon unsupported on MI300A) |
| Which functions/lines cause cache misses? | ATT / <code>linex</code> (§A.4/§H.4) | Valgrind <code>cachegrind</code> <code>cg_annotate</code> (§L) |
| How far from peak overall / project to another GPU? | roofline-extractor <code>--proj-arch</code> (§I) | n/a |
| Want the workflow driven & explained end-to-end? | rocBudAI AI assistant (§J) | rocBudAI (§J) |
| Why is SpMV memory-bound (which ISA lines stall)? | <code>rocprofv3 ATT (--att , §A.4)</code> ; IntelliKit <code>linex</code> (source-level SQTT, §H.4) | n/a |
| Did my SpMV optimization stay correct / faster? | IntelliKit <code>accordo validate + kerncap extract/replay</code> (§H) | n/a |
| SDMA vs blit copies? | <code>rocprofiler-systems timeline + HSA_ENABLE_SDMA sweep</code> | n/a |
| Where does each rank stall (load imbalance)? | HPCToolkit call-path | HPCToolkit / TAU |

Flag names and event lists vary across releases. If a flag is rejected, check `rocprofv3 --help`, `rocprof-compute --help`, `rocprof-sys-run --help`, `tau_exec -help`, `hpcrun -L`, `likwid-perfctr -a`, and `AMDuProfCLI collect --help` on the loaded module.

9. perf_events security — implementation & demo plan

This chapter turns the [perf_events security reference](#) (§M, a distillation of the [kernel perf-security guide](#)) into something you can **run** on the CG example.

The demo is a **least-privilege access model**, not a "lower the paranoid knob" hack. Concretely:

1. Create a `perf_users` group and add the users who are allowed to profile.
2. **Gate the profiling executables of ONE ROCm release** (plus the local `perf` ELF) to that group — `chgrp perf_users + chmod o-rwx` — and stamp `CAP_PERFMON` on them (`setcap`) where the filesystem supports file capabilities.
3. **Leave `perf_event_paranoid` open** (unchanged). Profilers from other ROCm releases are not capability-stamped, so they must keep relying on the open paranoid setting — tightening it would break them. The `perf_users + CAP_PERFMON` setup is a least-privilege **overlay on one release**, not a global change.

Two deliverables implement this:

| Script | Role | Privilege |
|--|---|--|
| CG-CPU/perf_users_setup.sh | Admin: create group, add users, gate + <code>setcap</code> one ROCm release's profilers | root (--apply);
dry-run by default |
| CG-CPU/perf_security_demo.sh | User: verify the posture (<code>paranoid</code> , group membership, caps) and exercise the permitted perf scopes on <code>cg_cpu</code> | none (read-only) |
| CG-CPU/perf_paranoid_test.sh | Proof: show an ordinary user can still profile under a raised <code>perf_event_paranoid</code> via <code>CAP_PERFMON</code> . Two modes: <code>verify</code> (no root; reads the prolog-set level) and <code>raise</code> (self-contained <code>sudo</code> , auto-restores) | none / <code>sudo</code> |
| CG-CPU/slurm/perf_paranoid.prolog.sh | Admin (SLURM): raise <code>perf_event_paranoid</code> per job when the user passes <code>--comment=paranoid[=N]</code> | installed as root; runs as root at job start |
| CG-CPU/slurm/perf_paranoid.epilog.sh | Admin (SLURM): restore the original <code>perf_event_paranoid</code> at job end | installed as root; runs as root at job end |

Goals

1. Grant profiling rights to a **named group**, not the whole node, via the kernel's `CAP_PERFMON` least-privilege capability (preferred over `CAP_SYS_ADMIN` and over blanket `paranoid=-1`).
2. Pilot the secure setup on **one ROCm release** while **every other release keeps working unchanged**.
3. Keep it safe to demonstrate: the admin script changes nothing unless run with `--apply` as root; the user script is entirely read-only.

Can we demo on one ROCm version without blocking on others? — Yes, by design

This is the whole point of leaving `paranoid` open:

- `perf_events` security is a **kernel** feature, identical under every `rocm/*` module. So the group/capability mechanism is version-independent.
- We stamp `CAP_PERFMON` on the profilers of **one** release (`rocm/7.2.3` here, a stable pick — note it predates and therefore lacks the ATT decoder, underscoring that perf security is independent of that GPU feature) and gate them to `perf_users`.
- Because `perf_event_paranoid` is **left open**, the **other** ROCm releases — whose profilers are not stamped — continue to work exactly as before. Nothing about them changes.

So you can roll the secure posture out **one release at a time** with zero risk to users on the other releases. The only genuinely ROCm-version-gated item anywhere in this chapter is the GPU ATT/SQTT decoder (§A.4/§H.4, needs `rocm >= 7.12`), which is unrelated to perf security.

What actually depends on the kernel (not ROCm). `CAP_PERFMON` needs Linux ≥ 5.9 ; this cluster is on 6.8. And **file capabilities need a supporting filesystem**: the local `perf` ELF lives on **xfs** (`setcap` works), but ROCm is on **NFS**, which cannot hold `security.capability` xattrs — so ROCm-release profilers are gated to the group but get `CAP_PERFMON` via the guide's `capsh` privileged shell (or simply via the open `paranoid`). The setup script detects the filesystem per target and does the right thing.

Implementation plan

Status: implemented and dry-run-verified (kernel 6.8, `rocm/7.2.3`, `/nfsapps` = NFS, `perf` ELF on xfs). The admin script `perf_users_setup.sh` performs:

1. **Group + membership.** `groupadd -f perf_users`; `usermod -aG perf_users <u>` for each `--users`. (Members must re-login for the group to take effect.)
2. **Target discovery.** The local `perf` **ELF** (resolved through the Ubuntu `/usr/bin/perf` wrapper to `/usr/lib/linux-tools/$(uname -r)/perf`) plus the `--profilers` found under

`--rocm/bin` (default: `rocpfv3`, `rocpfv-compute`, `rocpfv-sys-run`, `rocpfv-sys-sample`, `rocpfv-sys-instrument`).

3. **Gate + capability-stamp, per filesystem.** For each target: `chgrp perf_users` + `chmod o-rwx` (restrict to the group), then — only where `stat -f /findmnt` show a cap-capable, non-nosuid FS — `setcap cap_perfmon,cap_syslog=ep`. On NFS it prints the `capsh/open-paranoid` fallback instead of a broken `setcap`.
4. **Leave paranoid alone.** It prints the current `perf_event_paranoid` and an explicit note that it is **not** modified, so other releases keep working.

The **proof** that the overlay actually works is a separate step, `perf_paranoid_test.sh`, run once under `sudo` inside an **exclusive** allocation. It:

1. Saves the current `perf_event_paranoid` and installs a restore trap.
2. **Raises** it (default `2`; try `3` for stricter) — i.e. tightens security.
3. Runs perf **as the ordinary \$SUDO_USER** (via `runuser`, not as root): a per-process probe and a system-wide `perf stat -a` probe.
4. **Verdict:** if the system-wide probe passes at the raised level and the perf ELF carries `CAP_PERFMON`, the overlay is proven — an unprivileged user profiled a tightened node purely by capability, not by root. If it is denied, it tells you to run `perf_users_setup.sh --apply` first.
5. **Restores** the original `perf_event_paranoid` on exit (trap covers INT/TERM).

This is the crux of the demo: it shows the least-privilege grant is real and survives a security tightening — while, back at the normal open setting, every other ROCm release is unaffected.

Preferred: raise paranoid in the SLURM prolog/epilog (`--comment=paranoid`)

Rather than hand each tester `sudo` to poke the `sysctl`, do the privileged change **once, in the job prolog**, gated on a job comment. The user opts in per job with `--comment=paranoid[=N]`; the prolog raises `perf_event_paranoid`, the epilog restores it. Two drop-ins implement this (install them as root):

- `slurm/perf_paranoid.prolog.sh`
- `slurm/perf_paranoid.epilog.sh`

How the comment maps to a level (deployed form is `paranoid<num>`, no `=`; parsing verified against the `$M` scope ladder):

| <code>--comment=</code> | <code>perf_event_paranoid</code> | Effect on an unprivileged user |
|-------------------------|----------------------------------|--|
| <code>paranoid</code> | 2 (default) | per-process, user-space only; system-wide <code>-a</code> needs <code>CAP_PERFMON</code> |
| <code>paranoid1</code> | 1 | per-process (user+kernel); no system-wide |
| <code>paranoid2</code> | 2 | per-process, user-space only |
| <code>paranoid3</code> | 3 | vendor-strict: essentially nothing without <code>CAP_PERFMON</code> |
| <code>paranoid-1</code> | -1 | fully open (only useful to reset on a restricted node) |

| --comment= | perf_event_paranoid | Effect on an unprivileged user |
|------------|---------------------|--------------------------------|
| (none) | unchanged | job untouched — no-op |

paranoid=N and bare paranoid are also accepted. The comment may be embedded in a list (foo,paranoid2,bar); out-of-range values are rejected and logged; paranoidish/notparanoid do **not** match. The prolog **stashes** the pre-job value (/run/perf_paranoid.\$SLURM_JOB_ID) so the epilog restores it exactly; if the stash is lost it falls back to PERF_PARANOID_DEFAULT (default -1, the open value these exclusive compute nodes normally run). An optional **raise-only** guard (commented out in the prolog) refuses to lower security via --comment.

Install (as root, sudo): point slurm.conf at them (or drop into your existing prolog.d/epilog.d):

```
# /etc/slurm/slurm.conf    (then: scontrol reconfigure)
Prolog=/etc/slurm/perf_paranoid.prolog.sh
Epilog=/etc/slurm/perf_paranoid.epilog.sh
```

With that in place testers need **no sudo**: --comment=paranoidN raises it, and perf_paranoid_test.sh (verify mode) confirms profiling still works. The self-contained sudo ./perf_paranoid_test.sh N remains as a fallback where the prolog is not installed.

Verified (ppac-pl1-s24-16, kernel 6.8): srun --exclusive --comment=paranoid2 brought the node up at perf_event_paranoid = 2; per-process profiling worked (perf auto-scoped to :u), system-wide -a was denied (no CAP_PERFMON yet), and rocm/7.2.3's rocprofv3/rocprof-compute ran unaffected. One gotcha: SLURM_JOB_COMMENT is **not exported into the job step**, so the prolog reads the comment via its scontrol show job fallback — that fallback is essential, don't drop it.

Do the ROCm 7.2.3 profilers still work at paranoid=2? — Yes (verified)

The GPU profilers use rocprofiler-sdk / the KFD, **not** perf_events, so perf_event_paranoid does not gate them. Verified end-to-end on cg_gpu (gfx942, MI300A) inside --comment=paranoid2 with [CG-GPU/prof_all_723_test.sh](#):

| Profiler (ROCm 7.2.3) | Result at paranoid=2 | Notes |
|-------------------------------|---------------------------|--|
| rocprofv3 --sys-trace | PASS | clean |
| rocprofv3 --pmc (HW counters) | PASS | clean |
| rocprof-compute profile | PASS
(counters) | GPU counter collection OK; the optional empirical roofline microbench fails because the Nuitka onefile scrubs LD_LIBRARY_PATH for its bundled helper (libamdhip64.so.7 not found) — unrelated to paranoid |

| Profiler (ROCM 7.2.3) | Result at paranoid=2 | Notes |
|-----------------------|-----------------------------|---|
| rocprof-sys-sample | PASS (trace written) | writes the full Perfetto trace, then exits 134 on a glibc double-free at teardown (rocprofiler-systems v1.3.0 bug) — unrelated to paranoid |
| rocprof-sys-run | PASS (trace written) | same teardown double-free |

None emitted a `perf_event_paranoid` message. Two setup gotchas the script handles (neither is about paranoid):

1. **GPUs need `--gres`**. An `--exclusive` job alone hides `/dev/dri` in the cgroup, so `rocminfo` sees 0 GPUs and every GPU app aborts with "No HIP capable device". Request `--gres=gpu:4` (or `:1`).
2. **Use the patched `rocprof-compute`**. It ships as a self-contained **Nuitka single-file executable** under `rocm-patches-7.2.3/rocprof-compute/bin` (bundles pandas/dash/matplotlib/...). The plain `rocm-7.2.3/bin/rocprof-compute` is a Python script that fails on missing deps — so keep the `rocm-patches` path **ahead of** `rocm-7.2.3/bin` on `PATH` (the site module swap can leave `PATH/ROCM_PATH` on the 7.2.4 default; the test script pins 7.2.3 explicitly).

Design rules:

- **Dry-run by default** — prints every command, changes nothing; `--apply` requires root (guarded).
- **Filesystem-aware** — never emits a `setcap` that would fail on NFS.
- **Least privilege** — `cap_perfmon, cap_syslog` only (not `cap_sys_admin`; `cap_syslog` is for `/proc/kallsyms` kernel-symbol resolution). Add `cap_ipc_lock` only if you need `perf top`.
- **Idempotent-ish** — `groupadd -f`, `usermod -aG`, and `re-setcap` are safe to re-run.

Verified dry-run (excerpt)

```

1) Create group and add users
  groupadd -f 'perf_users'
  usermod -aG 'perf_users' 'alice'
2) Gate + capability-stamp the profiling executables
  target: /usr/lib/linux-tools/6.8.0-.../perf
  chgrp 'perf_users' '.../perf'
  chmod o-rwx '.../perf'
  setcap 'cap_perfmon, cap_syslog=ep' '.../perf'      # xfs -> works
  target: /nfsapps/.../rocm-7.2.3/bin/rocprofv3
  chgrp 'perf_users' '.../rocprofv3'
  chmod o-rwx '.../rocprofv3'
  # NFS: no file-capability support -> use capsh shell or open paranoid
3) perf_event_paranoid – LEFT UNCHANGED on purpose
  # current perf_event_paranoid = -1 (NOT modified)

```

Demo plan (how to present it)

```
# 0. (Admin, once) install the SLURM prolog/epilog so testers need no sudo:
sudo cp CG-CPU/slurm/perf_paranoid.prolog.sh /etc/slurm/
sudo cp CG-CPU/slurm/perf_paranoid.epilog.sh /etc/slurm/
# set Prolog=/etc/slurm/perf_paranoid.prolog.sh, Epilog=... ; scontrol reconfigure

# 1. Get an exclusive node AND ask the prolog to tighten paranoid for this job:
salloc -p PPAC_MI300A_SPX --exclusive -t 00:20:00 --comment=paranoid2
module load rocm/7.2.3 openmpi
cd CG-CPU && make CXXFLAGS="-O3 -g -std=c++17"

# 2. Show the overlay plan (dry-run, no changes), then apply it once:
./perf_users_setup.sh --users "$USER" --rocm "$ROCM_PATH"
sudo ./perf_users_setup.sh --apply --users "$USER" --rocm "$ROCM_PATH"

# 3. From a fresh login (so group membership is live), verify + exercise:
./perf_security_demo.sh

# 4. PROVE it (no sudo): the prolog already set paranoid=2 for this job.
./perf_paranoid_test.sh # verify mode: profiles as you via CAP_PERFMON
# Fallback where the prolog is NOT installed:
# sudo ./perf_paranoid_test.sh 2 # raise/restore it yourself
```

Talking points:

1. **Step 1** shows the whole privileged change up front — a group, some `usermod s`, and per-file `chgrp/setcap`. Nothing global; `perf_event_paranoid` is explicitly left untouched.
2. **xfs vs NFS** — the `perf` ELF gets a real `CAP_PERFMON`; the NFS-resident ROCm profilers are gated to the group and fall back to `capsh/open paranoid`. This is the practical reality on most clusters and the demo names it.
3. **Step 3** (`perf_security_demo.sh`) prints `perf_users member = yes` and `getcap (perf ELF) = ...cap_perfmon+ep`, then runs per-process and system-wide `perf` on `cg_cpu` — the memory-bound CPU signature (ties to \$K).
4. **Step 4** (`perf_paranoid_test.sh`) is the proof: at the tightened level, an ordinary user still lands a system-wide `perf stat -a` because the `perf` ELF carries `CAP_PERFMON`. Preferably the **prolog** raised `paranoid` (via `--comment=paranoid2`), so the tester needs no root at all; the self-contained `sudo` mode is a fallback and runs `perf` as `$SUDO_USER`, never as root.
5. **The version-independence payoff**: switch to any other `rocm` module and its profilers still work — because `paranoid` stayed open — even though only the 7.2.3 profilers carry the new group/cap posture.

Validation checklist

- [x] `perf_users_setup.sh`: `bash -n` clean; dry-run verified end-to-end.

- [x] Filesystem detection correct: xfs perf ELF → setcap; NFS ROCm → gate + fallback note.
- [x] Never modifies perf_event_paranoid; prints current value + rationale.
- [x] --apply refuses to run as non-root.
- [x] perf_security_demo.sh reports group membership + perf-ELF caps; read-only.
- [x] perf_paranoid_test.sh: bash -n clean; **verify mode** runs with no root (verified live on the login node at paranoid=4 — both probes correctly DENIED, sysctl untouched); raise mode's non-root path is the sudo fallback.
- [x] Prolog/epilog: bash -n clean; comment parser verified for paranoidN/paranoid=N/bare/embedded/out-of-range/paranoidish (no false match).
- [x] **End-to-end verified** on ppac-pl1-s24-16 (kernel 6.8.0-134): an exclusive srun --comment=paranoid2 job came up with perf_event_paranoid = 2 (login = 4), per-process perf stat ./cg_cpu-style profiling worked (auto-scoped to :u), system-wide -a was correctly denied (no CAP_PERFMON), and rocm/7.2.3's rocprofv3/rocprof-compute ran fine (unaffected by paranoid). NB: SLURM_JOB_COMMENT is **unset inside the job step**, so the prolog's scontrol show job fallback is what reads the comment — keep it.
- [] (Manual, needs admin) after perf_users_setup.sh --apply, re-run the same job and confirm the system-wide -a probe also PASSes via CAP_PERFMON.
- [] (Manual, needs admin) --apply on a real node, then re-login and confirm perf_security_demo.sh shows perf_users member = yes and the perf ELF caps.

See also

- [§M — perf_events security & access control](#) — the reference.
- [§K — Linux perf](#) — the profiler this secures.
- [Kernel perf-security guide](#) — the upstream source (Privileged Perf users groups, CAP_PERFMON, resource control).